Machine Learning for Customer Churn Prediction

Nelson David Posso Suarez
Javier Alejandro Penagos Hernandez
Nicolas Castro Rivera

1 Executive Summary

Customer Churn is a problem about customer discontinue a company's services or products. Machine learning techniques can be applied with purpose of predict if a customer has probability of be churn, this information can help the company to take decisions for not lost clients. This work search explore different machine learning techniques to compare and get the best approach to solve this problem.

2 Dataset Download and Integrity Verification

2.1 Dataset Source and Acquisition

The dataset is the **WSDM – KKBox Music Service Churn Prediction Challenge**, publicly available through the Kaggle competition platform at
`https://www.kaggle.com/competitions/kkbox-churn-prediction-challenge`.
The dataset was released by KKBox Co., Ltd. under the competition's terms of use, which permit academic and non-commercial research purposes. No paid subscription is required; a free Kaggle account is sufficient to access all files. To reproduce this experiment, the following two steps are required:

1. **Accept the competition terms:** Create a free Kaggle account, navigate to the competition page, and accept the data usage rules. This step is mandatory before any file can be downloaded.
2. **Download and place the raw files:** Download all nine `.csv.7z` archives provided by the competition and place them, **without extracting**, in the directory `workshops/data/`. The preprocessing pipeline extracts and converts them automatically during execution.

No further manual setup is required. The data pipeline reads the compressed archives directly from `workshops/data/` and writes the intermediate `.parquet` files to `workshops/data/parquet/`. The final processed cohorts are produced in `workshops/w1/export-tabular-v5/`.

2.2 Dataset Characteristics

The competition provides nine compressed files (`.csv.7z`). Table 1 summarises each file, its role in the pipeline, and its approximate uncompressed row count.

File	Description	Rows (approx.)	Columns
train.csv	February cohort labels: user ID and churn target	992,931	2
train_v2.csv	March cohort labels: user ID and churn target	970,960	2
members_v3.csv	User demographic profiles (age, gender, city, etc.)	6,769,473	7
transactions.csv	Earlier subscription transaction records	19,435,022	9
transactions_v2.csv	Recent subscription transaction records	21,591,521	9
user_logs.csv	Earlier daily listening behaviour logs	18,396,362	9
user_logs_v2.csv	Recent daily listening behaviour logs	18,396,362	9
sample_submission_v2.csv	Test set user IDs for competition submission	224,000	2
sample_submission_zero.csv	Baseline submission with all predictions set to zero	224,000	2

Table 1: KKBox dataset files and their characteristics.

2.3 Integrity Verification

The Kaggle platform does not publish official checksums for competition datasets. To guarantee that collaborators download the correct files, SHA-256 hashes were computed directly on the nine `.csv.7z` archives using the project script `workshops/w1/src/data_info/dataset_info.py`. The full hashes are stored in `workshops/data/checksums.json`; Table 2 shows the first 16 characters of each digest together with the compressed file size on disk.

File	SHA-256 (prefix)	Size (MB)
train.csv.7z	e2e7cec3392158ad	33.6
train_v2.csv.7z	683158156862e226	32.8
members_v3.csv.7z	a0e0b92b2ef9cc11	242.3
transactions.csv.7z	ff273f50f8e690c7	707.5
transactions_v2.csv.7z	78d01c8e90b86479	48.9
user_logs.csv.7z	2d21a83ddeeaf508	7,136.1
user_logs_v2.csv.7z	5e7b1d2b6fce3ff0	686.0
sample_submission_v2.csv.7z	329dfe694ccc76c7	30.7
sample_submission_zero.csv.7z	8d709c30a1e7dc52	32.8

Table 2: SHA-256 checksums of the raw `.csv.7z` archives (first 16 hex characters shown).

To verify the downloaded files, run the following command from the project root:

```
python workshops/w1/src/data_info/dataset_info.py
```

The script computes the SHA-256 of each archive in `workshops/data/`, compares the result against `checksums.json`, and prints `OK` or `FAIL` for each file. It exits with code 1 if any file is missing or does not match.

3 Exploratory Data Analysis

The KKBox dataset is composed of four distinct relational tables. Tables 3–6 describe the features present in each raw file, including a brief description and data type as stored in the source CSV.

Feature	Description	Data type
msno	Hashed subscriber identifier (primary key)	`str`
is_churn	Churn label: 1 if the user did not renew within 30 days of expiry, 0 otherwise	`int8`

Table 3: Schema of `train.csv` and `train_v2.csv`

Feature	Description	Data type
msno	Hashed subscriber identifier (primary key)	`str`
city	City of residence encoded as an integer code (1–22)	`int8`
bd	Subscriber age in years. Contains invalid values outside the range [1, 100]	`int16`
gender	Subscriber gender (male / female). Contains missing values	`str`
registered_via	Registration channel encoded as an integer code	`int8`
registration_init_time	Account registration date in YYYYMMDD format	`int32`
expiration_date	Membership expiration date in YYYYMMDD format. Not used as a model feature	`int32`

Table 4: Schema of `members_v3.csv`

Feature	Description	Data type
msno	Hashed subscriber identifier (foreign key)	`str`
date	Listening date in YYYYMMDD format	`int32`
num_25	Number of songs played less than 25% of their duration	`int32`
num_50	Number of songs played between 25% and 50% of their duration	`int32`
num_75	Number of songs played between 50% and 75% of their duration	`int32`
num_985	Number of songs played between 75% and 98.5% of their duration	`int32`
num_100	Number of songs played at least 98.5% of their duration (completed)	`int32`
num_unq	Number of unique songs played on that day	`int32`
total_secs	Total seconds of music listened to on that day	`float64`

Table 5: Schema of `user_logs.csv` and `user_logs_v2.csv`

Feature	Description	Data type
msno	Hashed subscriber identifier (foreign key)	`str`
payment_method_id	Payment method encoded as an integer code (2–41)	`int8`
payment_plan_days	Number of days in the purchased subscription plan	`int16`
plan_list_price	Listed price of the subscription plan in New Taiwan Dollars (NTD)	`int16`
actual_amount_paid	Amount actually paid by the subscriber in NTD	`int16`
is_auto_renew	Flag indicating automatic renewal (1 = yes, 0 = no)	`int8`
transaction_date	Date of the transaction in YYYYMMDD format	`int32`
membership_expire_date	Membership expiration date after the transaction in YYYYMMDD format	`int32`
is_cancel	Flag indicating subscription cancellation (1 = yes, 0 = no)	`int8`

Table 6: Schema of `transactions.csv` and `transactions_v2.csv`

The analysis is organised as a series of ten notebooks in `workshops/w1/eda/` (index in the folder's `README.md`). Notebooks 01–06 explore the raw tables on the labelled March cohort (`train_v2`); notebooks 07–10 explore the 59 engineered features on the February development cohort (`train_feb`, 992,931 customers), which is the data used to fit and tune the models. The figures in this section are produced by notebooks 07–10 and saved in `workshops/w1/figures/eda/`. All findings are descriptive: an association with churn is not a causal effect.

3.1 Target Distribution

In the February development cohort 63,471 of 992,931 customers churn (6.39%), a majority-to-minority ratio of 14.64:1 (Figure 1). In the March test cohort the rate is higher, 8.99% (87,330 of 970,960 customers). The target is therefore strongly imbalanced, and the imbalance changes between months.

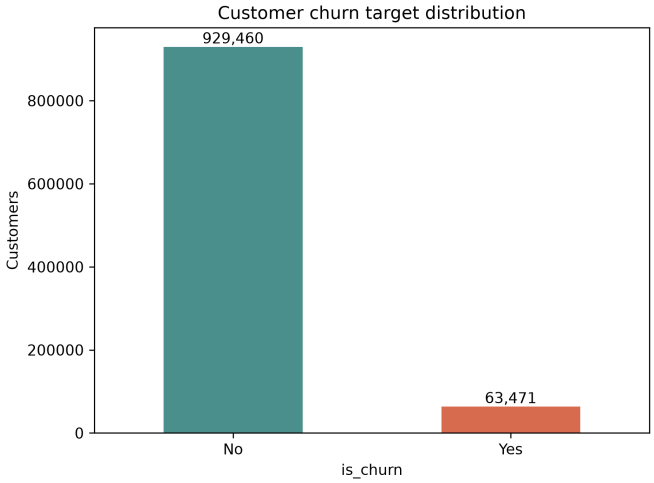


Figure 1: Target distribution in the February development cohort (`is_churn`).

Two consequences shape the rest of the work. First, accuracy is uninformative: always predicting "stays" is 93.6% accurate in February and 91.0% accurate in March. The models are instead compared with log-loss, average precision (AP), and the precision, recall, F1 and Matthews correlation coefficient (MCC) of the churn class. Second, the relationship plots show the churn rate within each group against the overall rate (dashed line) rather than raw counts.

3.2 Feature Statistics

The final table has 59 features: 29 from transactions, 22 from the listening logs and 8 from the member table. Each feature is assigned a type that decides which association measure is used (Table 7): Pearson, Spearman and the univariate ROC AUC for numeric features, Cramér's V for nominal codes, and churn rate per value or per decile for flags, counts and continuous features.

Source	Binary	Categorical	Constant	Continuous	Discrete	Total
Member	2	2	0	2	2	8
Transactions	6	1	1	15	6	29
Listening logs	1	0	0	21	0	22
Total	9	3	1	38	8	59

Table 7: Engineered features by source and type. The three nominal codes are `city`, `registered_via` and `last_payment_method_id`.

Many activity and payment aggregates are heavily right-skewed, so the median and the 99th percentile describe them better than the mean and maximum. The exploration of the raw tables (notebooks 02–05) adds the following observations:

- **Members.** There are 21 cities and city 1 dominates. Age (`bd`) contains negative values and values above 100, and its median is 0 because a missing age was stored as 0. Gender is almost entirely

empty, so it is not used as a feature. Registration has 18 channels, with codes 3, 4, 7 and 9 the most frequent.

- **Transactions.** Almost every plan lasts 30 days, although some plans reach about 450 days. The list price and the amount paid are almost identical; where they differ, the difference concentrates at 30, 149 and 180 and sometimes the customer pays more than the list price. Auto-renew is active in most transactions and cancellations are infrequent.
- **Listening logs.** The play counters have very long tails, and values above 1,200 songs in a day are implausible. Listening is bimodal: songs are either skipped before 25% or played to the end. The number of records per user is also bimodal (very few or many).
- **Dates.** Registrations grow strongly from year to year, and transactions and expiry dates follow the 30-day renewal cycle. Time therefore carries information and is encoded as features such as tenure, recency and days to expiry.

3.3 Data Quality

- **No member record.** 115,770 customers (11.7%) have no member record, so `city`, `tenure_days` and the registration fields are null for them. The flag `has_member_record` marks them.
- **No transactions.** 2,095 customers (0.2%) have no transaction history and their transaction statistics are null. They are a very high-risk group: 31.6% of them churn, almost five times the overall rate.
- **Invalid age.** The flag `bd_missing` is 1 for 603,791 customers (60.8%), whose raw age is invalid or absent. It is informative in itself: these customers churn *less* (4.8% versus 8.8%), so the flag is kept next to the imputed age.
- **Duplicates and invalid counters.** Exact duplicate transactions are removed and negative play counts are set to zero (see the preprocessing pipeline).

Missing values are kept in the analysis and dropped pairwise in each measure, so that no value is invented before modelling.

3.4 Feature Relationships

Numeric features. Figure 2 shows the Pearson and Spearman correlation of every numeric feature with churn. The strongest monotonic signals come from the subscription state: `last_is_auto_renew` ($\rho = -0.378$), `is_auto_renew_change_flag` (-0.348) and `autorenew_not_cancel` (-0.236) are negatively associated with churn, while `last_is_cancel` ($+0.270$) is positively associated. Longer plans and higher prices go with more churn (`last_plan_days` $+0.210$, `average_amount_charged` $+0.172$, `average_amount_paid` $+0.148$), whereas a longer payment history goes with less churn (`transaction_count` -0.142, `more_than_30` -0.139). The listening features are individually weak: none exceeds $|\rho| = 0.05$.

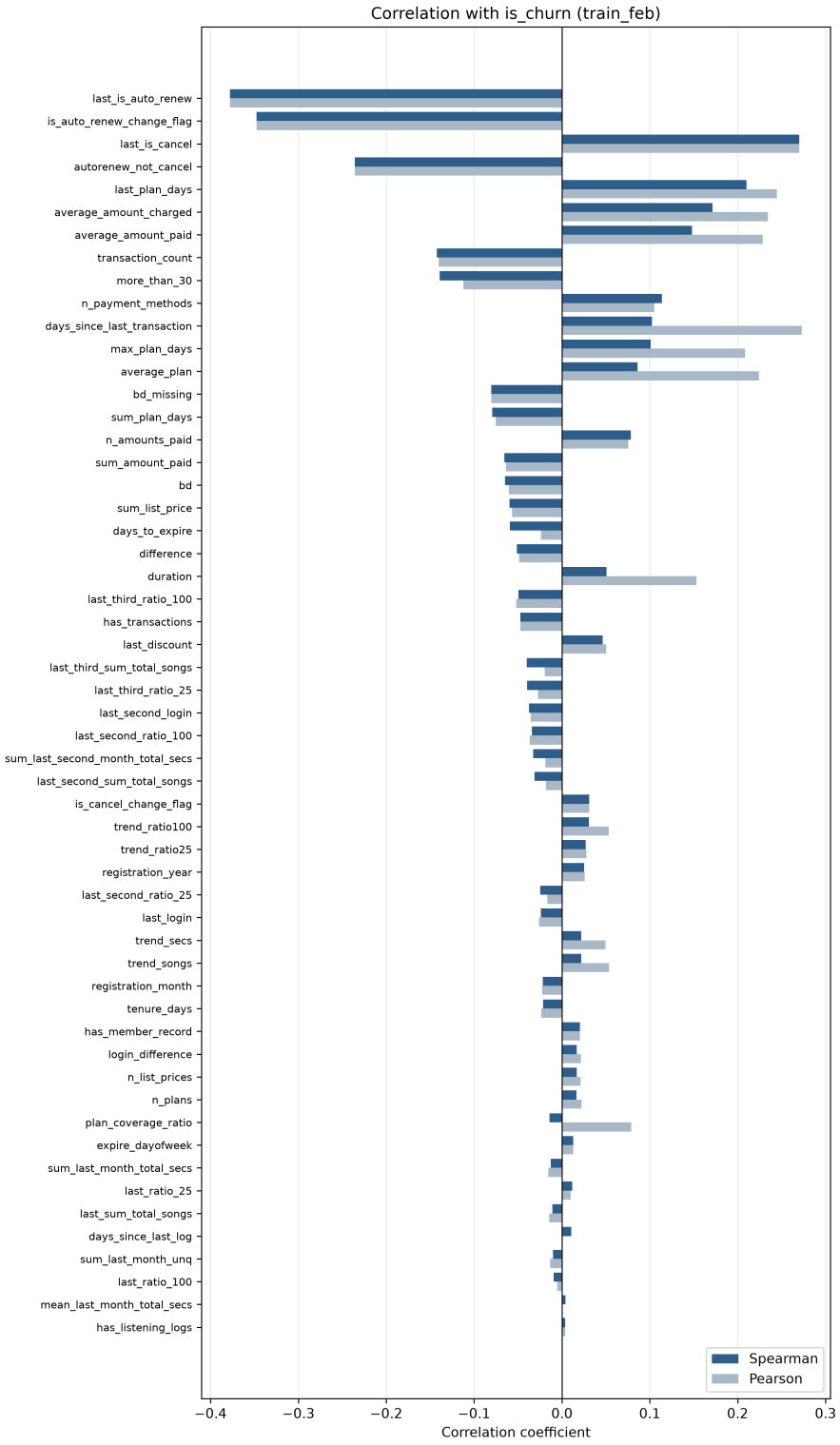


Figure 2: Spearman and Pearson correlation of each numeric feature with `is_churn` (February cohort), sorted by absolute Spearman correlation.

Nominal features. Cramér's V is 0.379 for `last_payment_method_id` (33 levels), 0.173 for `registered_via` and 0.073 for `city`. Figure 3 shows why: the payment method of the last transaction separates customers sharply (method 41, used by 570,014 customers, churns at 2.7%, method 38 at 30.5% and method 32 at 51.4%), `registered_via` ranges from 3.4% (code 7) to 18.3% (code 4), and

city 1 churns at 4.8% against 7.8–9.7% in the other large cities.

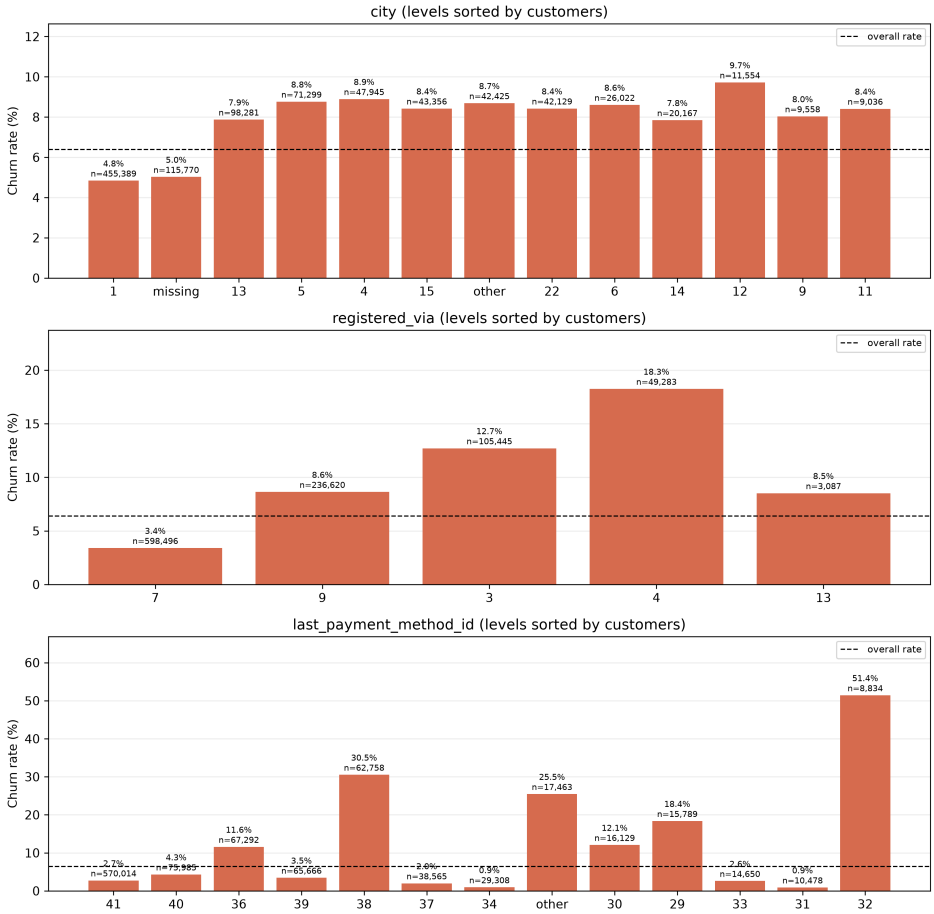


Figure 3: Churn rate per level of the three nominal features (most frequent levels; rarer levels pooled as `other`). The dashed line is the overall churn rate (6.39%).

Binary features. Figure 4 shows that auto-renew is the dominant flag: customers with `is_auto_renew_change_flag` = 0 churn at 31.7% (99,150 customers) against 3.5% when it is 1 (891,686 customers). The flags `has_member_record` (5.0% versus 6.6%) and `is_cancel_change_flag` (6.0% versus 7.8%) are much weaker.

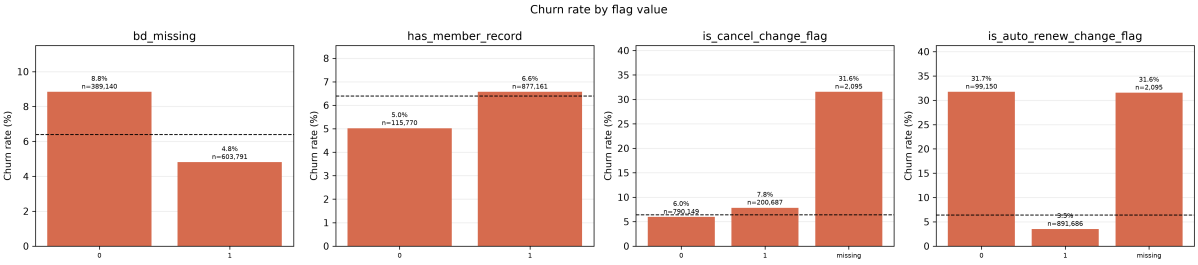


Figure 4: Churn rate by value of four binary features, annotated with the number of customers. The dashed line is the overall rate.

Continuous features. Figure 5 combines box plots by class (top) with the churn rate per decile (bottom), which exposes patterns that a correlation coefficient hides. `transaction_count` decreases steadily, from about 18% churn in the lowest decile (median of 2 transactions) to about 3% near 21

transactions. `average_amount_paid` is nearly flat at 3–6% and then rises to about 21% in the top decile (median 180), a threshold-like pattern. `bd` is highest for the youngest customers, and `tenure_days` shows no monotonic pattern. Several relationships are therefore non-linear, which favours tree-based models over a linear one.

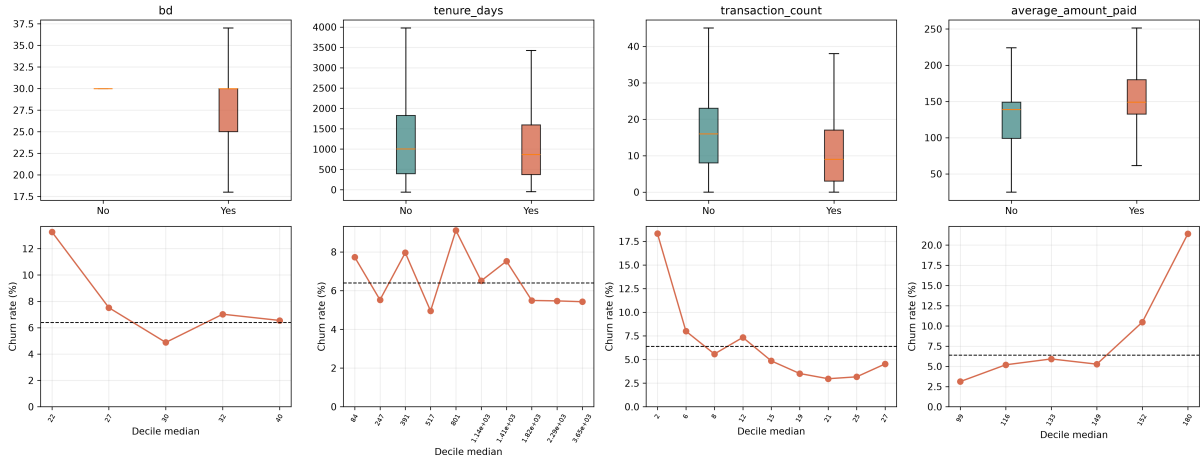


Figure 5: Distribution by class (top) and churn rate per decile (bottom) for four continuous features. The dashed line is the overall rate.

Redundancy. Many features are built from the same raw fields. Figure 6 shows the Spearman correlation between numeric features on a stratified sample of 200,000 customers; 21 pairs have $|\rho| \geq 0.9$, for example `sum_last_second_month_total_secs` with `last_second_sum_total_songs` (0.992), `registration_year` with `tenure_days` (-0.988), `sum_list_price` with `sum_amount_paid` (0.987) and `trend_songs` with `trend_secs` (0.980). Redundancy does not hurt tree ensembles but it affects the coefficients of the linear model.

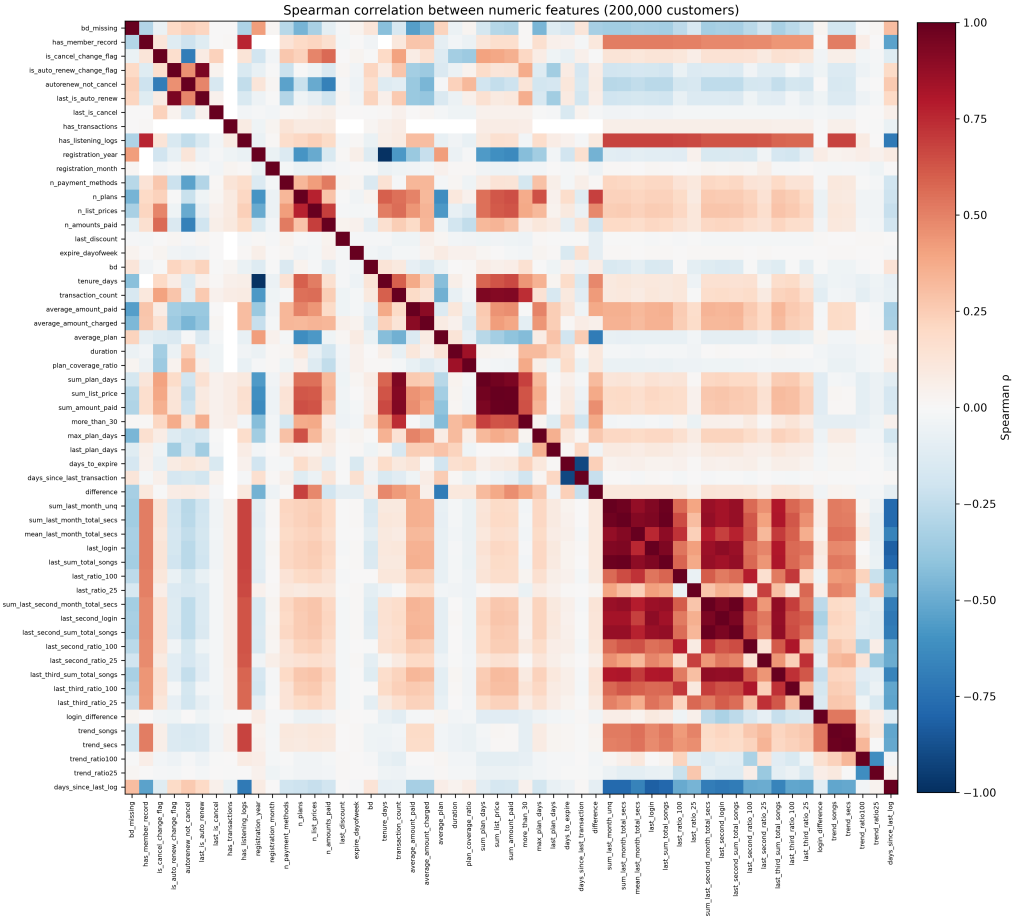


Figure 6: Spearman correlation between numeric features (stratified sample of 200,000 customers).

Unified ranking. Correlation coefficients and Cramér's V are on different scales, so the final ranking uses the mutual information with the target (Figure 7), estimated on the same sample. The 20 highest-ranked features are all transaction features except `bd` (rank 17); the top five are `last_plan_days`, `duration`, `last_payment_method_id`, `last_is_auto_renew` and `average_amount_charged`. The best listening feature, `days_since_last_log`, only reaches rank 24. The feature `plan_coverage_ratio` has $|\rho| = 0.014$ yet ranks ninth, an example of a non-monotonic relationship that correlation alone would discard.

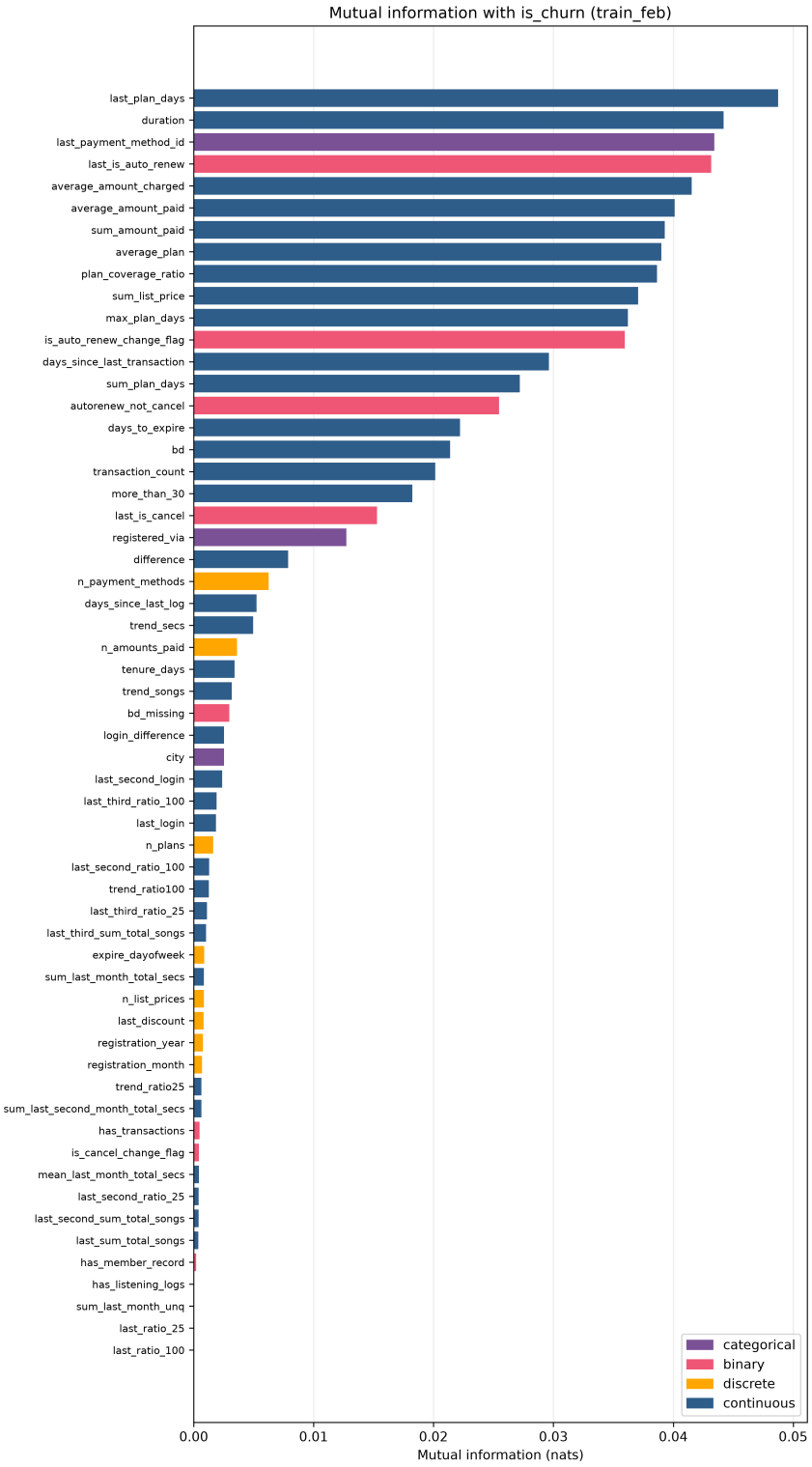


Figure 7: Mutual information (in nats) between each feature and `is_churn`, coloured by feature type.

Implications for modelling. (i) The strong and shifting imbalance requires threshold-independent metrics and an explicit decision threshold. (ii) The signal is concentrated in the subscription state (auto-renew, plan, payment method and payment history); listening behaviour adds little on its own. (iii) Non-linear and threshold-like patterns, together with categorical levels with very different rates, favour tree-based models. (iv) Redundant feature blocks call for regularisation in the linear model.

4 Preprocessing Pipeline

The preprocessing pipeline implemented in this work is partially based on the public analysis published by Asad Rizvi [1], which includes an exploratory data analysis and a feature engineering approach for the KKBox Churn Prediction Challenge. From his work we adopted the general structure for joining the relational tables and several feature engineering ideas at the transaction and listening level. However, his approach targeted a different modelling objective and proposed a larger feature set with a different temporal framing. We discarded a significant portion of his proposed features, simplified the listening aggregation scheme, and redesigned the cohort structure around a month-over-month temporal split that better reflects a real production scoring scenario.

The full pipeline is implemented in `workshops/w1/w1_data_preprocessing.ipynb`. The model training and evaluation that consumes the pipeline output is in `workshops/w1/w1_tabular_models.ipynb`.

4.1 Raw Extraction and Parquet Conversion

The nine `.csv.7z` archives in `workshops/data/` are extracted and converted to Parquet format by the `convert_raw_to_parquet()` function (`workshops/w1/src/data_processing/utils.py`), called in Section 2 of the preprocessing notebook. Each archive is decompressed one at a time using the `7z` binary, and only the columns defined in the project schema are retained and cast to compact integer or float types. The resulting files are written with `zstd` compression to `workshops/data/parquet/`. The conversion is idempotent: existing Parquet files are skipped, so the step can be safely resumed after an interruption.

Parquet was chosen over CSV because it is a columnar binary format that stores type information natively. Reading a single column from a large Parquet file does not require parsing the entire file, which reduces both I/O and memory usage significantly compared to CSV for the multi-million-row tables in this dataset.

4.2 Cohort Definition and User Index

The dataset is structured around two labelled cohorts defined in Section 3 of the preprocessing notebook:

- **February development cohort** (`train.csv`): 992,931 users with features cut at 2017-01-31 and churn labels for February 2017. Churn rate: 6.4%.
- **March test cohort** (`train_v2.csv`): 970,960 users with features cut at 2017-02-28 and churn labels for March 2017. Churn rate: 9.0%.

A shared integer index (`uid`) is assigned over the union of both cohorts (1,082,190 unique users), replacing the raw `msno` string for all subsequent joins. A user who appears in both months receives the same `uid` but a different cutoff, feature snapshot, and label, which mirrors monthly scoring in production.

4.3 Member Features

Static demographic features are extracted from `members_v3.parquet` in Section 4 of the preprocessing notebook. Invalid ages (outside the range [1, 100]) and negative registration channel values are imputed with the mean age and mode channel computed exclusively on the February cohort. Three derived features are added: `tenure_days` (days elapsed from the registration date to the feature cutoff), `registration_year`, and `registration_month`. A binary flag `bd_missing` marks rows where the original age was invalid or absent. This domain contributes **7 features** per user.

4.4 Transaction Features

Transaction records from both `transactions.parquet` and `transactions_v2.parquet` are concatenated and deduplicated in Sections 5–6 of the preprocessing notebook. Exact duplicates—rows identical across all eight transaction fields for the same user—are removed, keeping only the first occurrence. Per-record derived columns are then computed: plan duration, plan coverage ratio, discount (list price minus amount paid), and behavioural flags. These records are subsequently aggregated into **one row per user**, yielding approximately 29 features covering counts, sums, averages, last-record attributes, and timing features relative to the cutoff (e.g., `days_to_expire`, `days_since_last_transaction`). All transaction history up to the cohort cutoff is used; no fixed time window is applied.

4.5 Listening Log Features

Daily listening logs from both `user_logs.parquet` and `user_logs_v2.parquet` are processed in Sections 7–8 of the preprocessing notebook. Records are restricted to the three calendar months ending at the feature cutoff and filtered to target users. Negative play counts are set to zero and non-finite `total_secs` values are replaced by zero. The resulting user-day table is grouped to monthly sums and active-day counts. A daily p99.9 clipping limit is applied to the per-day mean of each listening counter—these limits are fitted exclusively on the February cohort and reused for March to prevent leakage.

The last three active months for each user are then pivoted into a wide row (`last_*`, `last_second_*`, `last_third_*`). Several redundant per-channel columns are dropped, and four month-over-month trend features are added: `trend_songs`, `trend_secs`, `trend_ratio100`, and `trend_ratio25`. Recency features (`days_since_last_log`, `has_listening_logs`) are computed relative to the cutoff. This domain contributes approximately **23 features** per user.

4.6 Final Assembly and Saved Outputs

The three feature domains are assembled in Sections 9–10 of the preprocessing notebook by left-joining member, transaction, and listening features onto the user list. Users absent from a given domain receive zero for all count and sum columns; presence flags (`has_member_record`, `has_transactions`, `has_listening_logs`) mark whether a record existed. Table 8 lists the resulting **59 features** per user together with their source table.

Feature	Description	Source
city	City of residence (integer code)	members_v3
registered_via	Registration channel (integer code)	members_v3
bd	Subscriber age in years (imputed if invalid)	members_v3
bd_missing	Flag: 1 if original age was invalid or absent	members_v3
tenure_days	Days from registration to feature cutoff	members_v3
registration_year	Year of account registration	members_v3
registration_month	Month of account registration	members_v3
has_member_record	Flag: 1 if a member record exists for the user	Pipeline
is_cancel_change_flag	1 if any transaction was a cancellation	transactions
is_auto_renew_change_flag	1 if any transaction had auto-renew enabled	transactions
transaction_count	Total number of transactions	transactions
average_amount_paid	Mean amount paid across all transactions	transactions
average_amount_charged	Mean listed price across all transactions	transactions
average_plan	Mean plan duration in days	transactions
n_payment_methods	Number of distinct payment methods used	transactions
n_plans	Number of distinct plan durations purchased	transactions
n_list_prices	Number of distinct listed prices seen	transactions
n_amounts_paid	Number of distinct amounts actually paid	transactions
duration	Mean duration (expiry minus transaction date) in days	transactions
plan_coverage_ratio	Mean ratio of duration to plan days	transactions
sum_plan_days	Total plan days purchased	transactions
sum_list_price	Total listed price across all transactions	transactions
sum_amount_paid	Total amount paid across all transactions	transactions
more_than_30	Count of transactions with duration > 30 days	transactions
autorenew_not_cancel	Min flag: auto-renew on and not cancelled	transactions
not_autorenew_cancel	Min flag: auto-renew off and cancelled	transactions

Continued on next page

Feature	Description	Source
max_plan_days	Maximum plan duration across all transactions	transactions
last_payment_method_id	Payment method of the most recent transaction	transactions
last_plan_days	Plan days of the most recent transaction	transactions
last_is_auto_renew	Auto-renew flag of the most recent transaction	transactions
last_is_cancel	Cancel flag of the most recent transaction	transactions
last_discount	Discount (list price minus paid) of the most recent transaction	transactions
days_to_expire	Days from last expiry date to cutoff (negative = already expired)	transactions
expire_dayofweek	Day of the week of the last expiry date	transactions
days_since_last_transaction	Days from last transaction date to cutoff	transactions
difference	Mean amount paid minus mean amount charged	transactions
has_transactions	Flag: 1 if any transaction record exists	Pipeline
sum_last_month_unq	Sum of unique songs in the last active month	user_logs
sum_last_month_total_secs	Sum of listening seconds in the last active month	user_logs
mean_last_month_total_secs	Mean daily listening seconds in the last active month	user_logs
last_login	Number of active days in the last active month	user_logs
last_sum_total_songs	Total songs played in the last active month	user_logs
last_ratio_100	Ratio of fully-played songs in the last active month	user_logs
last_ratio_25	Ratio of barely-played songs in the last active month	user_logs
sum_last_second_month_total_secs	Sum of listening seconds in the second-to-last active month	user_logs
last_second_login	Number of active days in the second-to-last active month	user_logs
last_second_sum_total_songs	Total songs played in the second-to-last active month	user_logs

Continued on next page

Feature	Description	Source
last_second_ratio_100	Ratio of fully-played songs in the second-to-last active month	user_logs
last_second_ratio_25	Ratio of barely-played songs in the second-to-last active month	user_logs
last_third_sum_total_songs	Total songs played in the third-to-last active month	user_logs
last_third_ratio_100	Ratio of fully-played songs in the third-to-last active month	user_logs
last_third_ratio_25	Ratio of barely-played songs in the third-to-last active month	user_logs
login_difference	Difference in active days between last and second-to-last month	user_logs
trend_songs	Ratio of total songs last month to second-to-last month	user_logs
trend_secs	Ratio of listening seconds last month to second-to-last month	user_logs
trend_ratio100	Change in fully-played ratio from second-to-last to last month	user_logs
trend_ratio25	Change in barely-played ratio from second-to-last to last month	user_logs
days_since_last_log	Days from the last listening date to the cutoff	user_logs
has_listening_logs	Flag: 1 if any listening record exists	Pipeline

Table 8: Final feature set (59 features) with source table.

The two cohort tables are exported to `workshops/w1/export-tabular-v5/` as `train_feb.parquet` (February, 992,931 users) and `test.parquet` (March, 970,960 users). Imputation and clipping parameters are persisted in `preprocess_params.json` and the complete feature catalogue in `metadata.json`. Split indices for the development and test partitions are saved to `workshops/w1/artifacts/sherlock_w1/split_indices.npz`.

5 Data Partitioning

The split protocol is `cross_month_feb_to_mar`: the model is developed on one month and evaluated on the next, which mimics deployment, where a model trained on past customers must score a new month. Two cohorts are exported by the preprocessing pipeline, with the same 59 features but different observation cutoffs (Table 9).

Part	Source	Customers	Churners	Churn rate	Use
fit	February (75%)	744,699	47,604	6.39%	train model parameters
stop	February (12.5%)	124,117	7,934	6.39%	early stopping (XGBoost)
tune	February (12.5%)	124,115	7,933	6.39%	Optuna objective, threshold, model selection
test	March (100%)	970,960	87,330	8.99%	final evaluation only

Table 9: Partition of the data. The February cohort (observation cutoff 2017-01-31) is split into fit/stop/tune; the March cohort (cutoff 2017-02-28) is the test set.

- **Deterministic assignment.** Each customer is assigned to a part from a SHA-1 hash of `msno` with a fixed salt, so the assignment does not depend on row order or on a random state, and it is reproducible across machines. It is stratified by the label, so the three February parts keep the same churn rate (6.39%).
- **Roles.** *fit* trains the parameters of each model. *stop* is used only by XGBoost to stop adding trees when the validation loss stops improving. *tune* (the validation set) is where Optuna evaluates hyperparameters, where the decision threshold is chosen and where the winning model is selected. Separating *stop* from *tune* prevents the early-stopping criterion from leaking into the hyperparameter comparison.
- **Test set.** March is never used to fit, tune or choose a threshold. It is used once, for the final metrics.
- **No leakage from preprocessing.** Imputation medians, scaling statistics and category encodings are learned on the fit/development data only and stored (`preprocessing/development.json`); the test cohort is transformed with those fixed values.
- **Final model.** After the hyperparameters and the threshold are chosen, each model is refitted on all of February (fit, stop and tune together, 992,931 customers) before scoring March.

Because the test set is a different month, it also measures robustness to a change of the churn rate (6.39% to 8.99%); this shift matters for the interpretation of the results.

6 Models and Hyperparameter Tuning

6.1 Baseline Models

Three models with increasing capacity are compared on the same 59 features and the same split.

- **M1 – Logistic regression.** Numeric features are median-imputed and standardised; the nominal codes are one-hot encoded (unseen categories ignored). It is fitted with the `newton-cholesky` solver with at most 200 iterations. It is an interpretable linear reference whose only tuned parameter is the inverse regularisation strength C.
- **M2 – Random Forest.** An ensemble of decision trees trained on bootstrap samples with random feature subsets [2]. It handles non-linear effects and interactions without scaling and is robust to the redundancy between features.
- **M3 – XGBoost.** Gradient-boosted trees [3] fitted on GPU with early stopping on the *stop* part (up to 3,000 rounds, patience 100). To favour recall, the positive class is up-weighted with `scale_pos_weight` equal to twice the natural ratio of negatives to positives (about 29.3 on the fit data).

6.2 Tuning with Optuna

Hyperparameters are optimised with Optuna [4] using the TPE sampler (seed 110) and a median pruner. The objective is the log-loss on the *tune* part, which is a proper scoring rule and does not depend on a threshold. Studies are stored in SQLite so that interrupted runs resume. The first trial of every study is a fixed, sensible starting configuration, so the search can only improve on it. Table 10 gives the search spaces and the best values found.

Model	Hyperparameter	Search space	Best value
M1	C	log-uniform $[10^{-4}, 10^{2}]$	91.1
M2	`n_estimators`	$\{100, 150\}$	150
	`max_depth`	$\{8, 10, 12, 16, 20, 24\}$	12
	`min_samples_leaf`	integer $[5, 100]$	5
	`min_samples_split`	integer $[10, 100]$	10
	`max_features`	$\{$`sqrt`, `log2`, $0.5\}$	`sqrt`
M3	`learning_rate`	log-uniform $[0.01, 0.15]$	0.0287
	`max_depth`	integer $[3, 10]$	10
	`min_child_weight`	uniform $[1, 30]$	5.72
	`subsample`	uniform $[0.6, 1.0]$	0.900
	`colsample_bytree`	uniform $[0.6, 1.0]$	0.610
	`reg_alpha`	log-uniform $[10^{-4}, 10]$	0.393
	`reg_lambda`	log-uniform $[10^{-4}, 10]$	0.0471
	`gamma`	log-uniform $[10^{-4}, 5]$	4.16
	(selected rounds)	early stopping	265

Table 10: Optuna search spaces and best configurations for M1–M3 (objective: log-loss on *tune*).

The compute budget was small: 3 trials for M1, 3 for M2 and 8 for M3 (the configured ceiling is 50). Table 11 shows the outcome. For M1 the log-loss differs only in the fourth decimal and the best C is close to the upper edge of the range, so a larger C (weaker regularisation) is not excluded. For M2 the best trial is the initial configuration, so the three trials did not find an improvement. For M3 the search lowered the tune log-loss from 0.527 (initial configuration) to 0.494. These studies are therefore a light tuning rather than an exhaustive search, and the differences between models should be read with this in mind.

Model	Trials	First trial	Best trial	Best tune log-loss
M1	3	0.15557	3rd	0.15537
M2	3	0.14485	1st	0.14485
M3	8	0.52699	6th	0.49402

Table 11: Optuna summary. "First trial" is the log-loss of the fixed starting configuration.

Probability scale of M3. The tune log-loss of M3 (0.494) is not comparable with M1 and M2 (0.155 and 0.145): up-weighting the churners makes XGBoost's scores systematically larger than the true probabilities, which inflates the log-loss. As a consequence, the F1-optimal threshold of M3 is 0.88, whereas it is 0.28 and 0.26 for M1 and M2. M3's scores rank customers well but must not be read as

probabilities. Table 12 therefore selects the winner on the tune log-loss among the models whose scores are probabilities, and the ranking metrics (ROC AUC, AP) are used to compare all three.

6.3 Decision Threshold and Model Selection

The threshold of each model is the value that maximises the F1 score of the churn class on *tune*. The default 0.5 is inappropriate here, because with 6% positives most calibrated probabilities fall below 0.5. The model with the lowest tune log-loss, M2 (Random Forest), is the primary winner (Table 12).

Model	Tune log-loss	F1-optimal threshold
M1 Logistic regression	0.1554	0.28
M2 Random Forest	**0.1449**	0.26
M3 XGBoost	0.4940	0.88

Table 12: Model selection on the tune part (lower log-loss is better).

7 Results

All figures below are computed on the March cohort (970,960 customers, 8.99% churn) with the models refitted on February. Table 13 reports the threshold-free metrics and the metrics of the churn class at each model's tuned threshold.

Model	Log-loss	Brier	ROC AUC	AP	Thr.	Prec.	Recall	F1	MCC
M1	0.2126	0.0564	0.8445	0.5553	0.28	0.581	0.534	0.557	0.515
M2	**0.2032**	**0.0548**	**0.8632**	**0.5933**	0.26	**0.638**	0.539	**0.584**	**0.549**
M3	0.4886	0.1498	0.8520	0.5882	0.88	0.609	**0.542**	0.574	0.536

Table 13: Test results on March. Precision, recall, F1 and MCC refer to the churn class at the threshold selected on *tune*. A classifier that predicts nobody churns has AP = 0.0899 (the churn rate); one that predicts everybody churns has F1 = 0.165.

Table 14 gives the confusion matrices at the tuned thresholds. Out of 87,330 churners, M2 detects 47,028 and misses 40,302, and raises 26,694 false alarms among 883,630 customers who stay.

Model	TN	FP	FN	TP	Predicted churn	Actual churn
M1	849,988	33,642	40,668	46,662	8.27%	8.99%
M2	856,936	26,694	40,302	47,028	7.59%	8.99%
M3	853,263	30,367	39,959	47,371	8.01%	8.99%

Table 14: Confusion matrices on March at the tuned thresholds (positive class: churn).

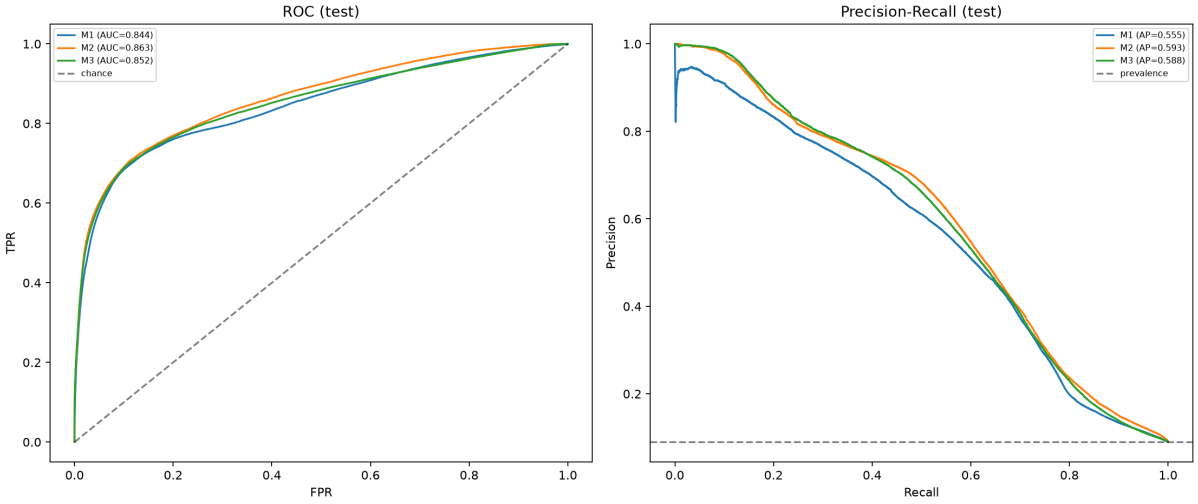


Figure 8: ROC and precision–recall curves of M1–M3 on the March test cohort.

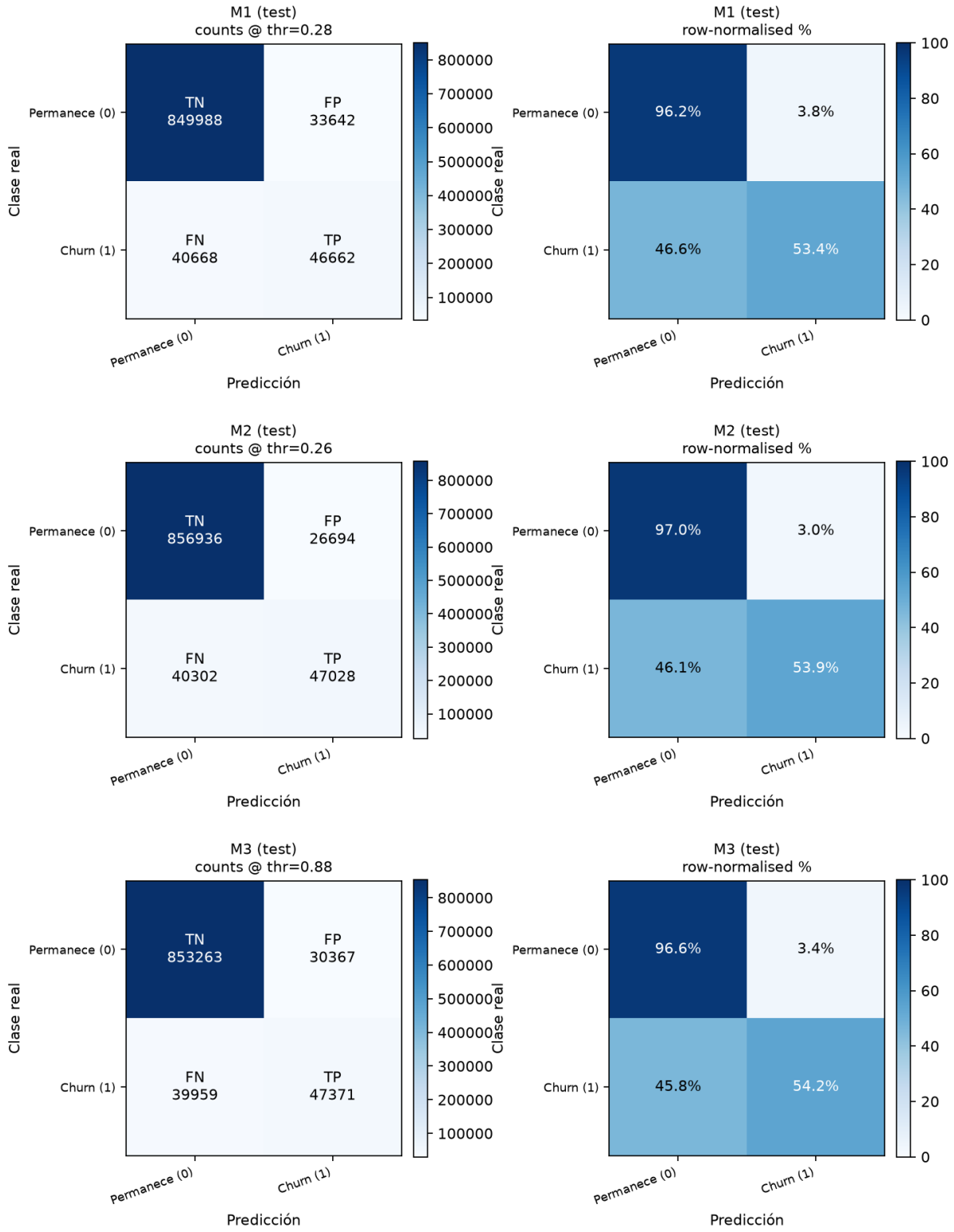


Figure 9: Confusion matrices on the March test cohort at the tuned thresholds.

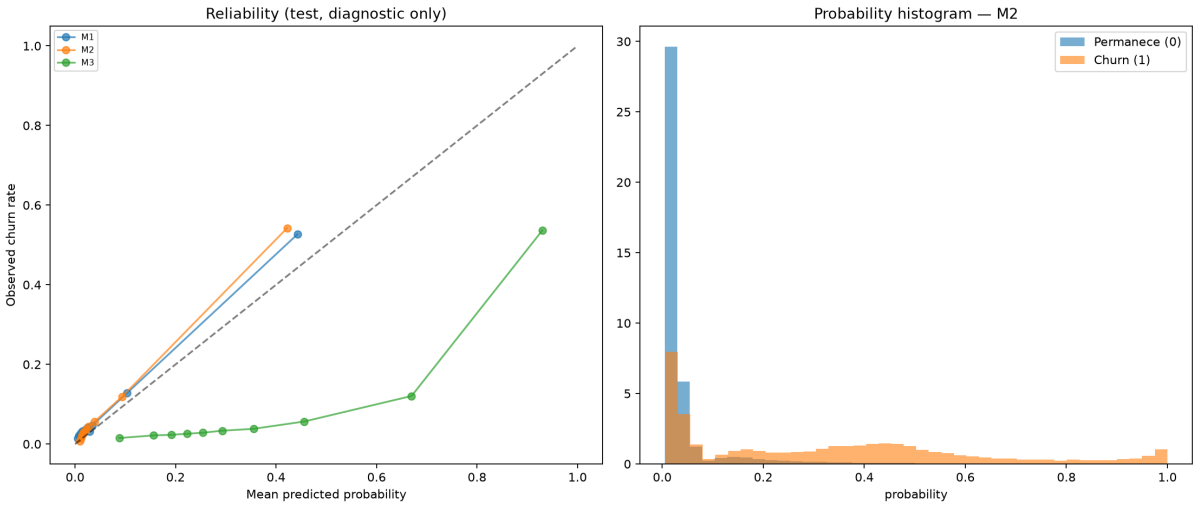


Figure 10: Calibration curves and distribution of predicted scores on the March test cohort.

7.1 Model Comparison

- **M2 (Random Forest) is the best overall.** It has the highest ROC AUC (0.863), AP (0.593), precision, F1 and MCC, and the lowest log-loss and Brier score of the three. It also gives the fewest false alarms.
- **M3 (XGBoost) is very close in ranking.** Its AP (0.588) and AUC (0.852) are within one point of M2, and it has the highest recall (0.542). Its probabilities are not calibrated because of the class weight, so its log-loss and Brier score are much worse and cannot be compared with the other two.
- **M1 (logistic regression) is the weakest** (AP 0.555, F1 0.557) but not by a large margin: a linear model on the engineered features is already competitive, which indicates that most of the signal is in a few strong features (auto-renew, plan, payment method).
- **The differences among models are small compared with the gap to the trivial baselines.** The best AP is 6.6 times the no-skill AP (0.0899), whereas the gap between M2 and M1 is 0.04. With only 3–8 tuning trials per model, ranking the three should be taken as indicative, not definitive.

7.2 Why the F1 Score Is Low

The best F1 for the churn class is 0.58, which looks low next to an accuracy above 92%. This is expected given the problem and does not by itself indicate a bug:

1. **Accuracy is not informative.** Predicting "stays" for everybody gives 91.0% accuracy in March. The accuracies of 92.3–93.1% obtained by the models are only about two points above this trivial classifier, while F1 focuses on the minority class.
2. **Class imbalance makes precision hard.** With 9% positives, a model that is right on 54% of the churners and wrong on 3% of the stayers (M2: recall 0.539, false-positive rate 0.030) has a

precision of only 0.64, because there are ten times more stayers than churners to make mistakes on. The precision–recall trade-off is also stiff: the AP of 0.59 means that precision drops quickly as recall grows (Figure 8).

3. **ROC AUC looks good while F1 does not.** An AUC of 0.86 means that a random churner is ranked above a random non-churner 86% of the time, but the ROC curve is insensitive to the number of negatives; AP and F1 are not. The metrics measure different things and are consistent with each other.

4. **The threshold was tuned on a month with a different prevalence.** The F1-optimal thresholds were selected on February (6.39% churn) and applied to March (8.99%). All three models therefore predict fewer churners than there actually are (7.6–8.3% against 9.0%), and the recall stays around 0.54. A threshold tuned on March itself would give a higher recall, so part of the loss is due to the prior shift between months (the same shift raises the log-loss from 0.145 on *tune* to 0.203 on the test set for M2).

5. **Limited information.** The features are aggregates of subscription, payment and listening behaviour up to the cutoff. Many customers leave for reasons not recorded in these tables (price, competition, life events), and the listening features carry little individual signal (Section 3.4); this puts a ceiling on the achievable F1 regardless of the model.

6. **Light tuning.** The small number of Optuna trials leaves some room for improvement, although the extra experiment below suggests that it is limited.

Table 15 shows the effect of the threshold directly. At the default 0.5, M1 and M2 detect only 24–27% of the churners (precision 0.78–0.83), while M3, whose scores are inflated, flags 21% of all customers; none of them is useful at 0.5, which justifies tuning the threshold.

Model	Precision	Recall	F1	Predicted churn
M1	0.780	0.273	0.405	3.1%
M2	0.828	0.241	0.373	2.6%
M3	0.312	0.741	0.439	21.4%

Table 15: Metrics of the churn class on March at the default threshold 0.5.

7.3 Additional Experiment: XGBoost v2

To check whether more tuning would change the picture, a second XGBoost study (M3v2) was run: 80 Optuna trials maximising the average precision on *tune*, a wider search space (growth policy, leaves, column sampling per node, maximum delta step and `scale_pos_weight` in $[1, 30]$), and isotonic calibration fitted on *tune*. Its best configuration uses `scale_pos_weight` $= 1.90$, i.e. almost no re-weighting. Table 16 shows that M3v2 reaches the best ROC AUC (0.864) and a log-loss (0.204) now comparable

with M2, but its AP (0.590) and its F1 at the F1-optimal threshold (0.576) are not better than M2's (0.593 and 0.584). A recall-oriented threshold ($\geq$0.70 on *tune*) gives a recall of 0.645 at a precision of 0.482 (F1 0.552).

Model / criterion	**ROC AUC**	**AP**	**Log-loss**	**Prec.**	**Recall**	**F1**
M2 (F1 threshold)	0.8632	0.5933	0.2032	0.638	0.539	0.584
M3 (F1 threshold)	0.8520	0.5882	0.4886	0.609	0.542	0.574
M3v2 (F1 threshold)	0.8642	0.5900	0.2037	0.659	0.511	0.576
M3v2 (recall $\geq$ 0.70 on tune)	0.8642	0.5900	0.2037	0.482	0.645	0.552

Table 16: XGBoost v2 compared with the main models on March. The F1 threshold of M3v2 is 0.275.

More tuning therefore improves calibration but not the ranking quality: the three tree models converge to an AP of about 0.59, which supports the interpretation that the remaining error is due to the information in the features and to the prior shift, not to the hyperparameters. M3v2 is reported as a complementary experiment; the selection protocol above (M1–M3, winner on *tune* log-loss) is unchanged.

7.4 Conclusions of Workshop 1

- The churn target is imbalanced (6.4% in February, 9.0% in March) and the signal is concentrated in subscription features, mainly auto-renew status, plan length, payment method and payment history.
- The partition (fit/stop/tune from February, March as test) keeps the test month untouched and measures robustness to a change of churn rate.
- The Random Forest (M2) is selected: ROC AUC 0.863, AP 0.593, F1 0.584 and MCC 0.549 on March. XGBoost is a close second and logistic regression is a competitive linear baseline.
- An F1 around 0.58 is a consequence of class imbalance, the precision–recall trade-off, the shift in churn rate between months and the limited information in the features; it is not evidence of a faulty model. The metrics must be read together (AP, MCC, recall and precision), never accuracy alone.
- Next steps: a larger Optuna budget, a threshold chosen with the expected prevalence of the target month, and sequence models that use the temporal structure of the logs (Workshop 2).

References

[1] Rizvi, A. (2018). *WSDM – KKBox's Churn Prediction Challenge*. Medium. Retrieved from `https://asad-99rizvi.medium.com/wsdm-kkboxs-churn-prediction-challenge-d7c9cfa21cbd`

[2] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[3] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[4] T. Akiba, S. Sano, T. Yanase, T. Ohta and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.